GHC.ST: https://hackage.haskell.org/package/base/docs/GHC-ST.html
GHC.STRef: https://hackage.haskell.org/package/base/docs/GHC-STRef.html

```
-- GHC.ST
newtype ST s a = ST (STRep s a)
data STret s a = STret (State# s) a
type STRep s a = State# s -> (# State# s, a #)

instance Functor (ST s)
instance Applicative (ST s)
instance Monad (ST s)
instance MonadFix (ST s)
instance Monoid a => Monoid (ST s a)

runST :: (forall s. ST s a) -> a
liftST :: ST s a -> State# s -> STret s a


-- GHC.STRef
newSTRef :: a -> ST s (STRef s a)
readSTRef :: STRef s a -> ST s a
writeSTRef :: STRef s a -> a -> ST s ()
modifySTRef :: STRef s a -> (a -> a) -> ST s ()
```

ST
A way to use a **mutable value** in a pure function internally.
Primarily for **optimization** i.e. some computations perform better with **mutable** values.

ST s a	computation ST in a **thread s** with **result a**
STRef s a	a **mutable variable** of **type a** in an ST **thread s**

ST s represents a **session s** for supporting mutable values.
A mutable value created in an ST persists only **within** the same ST i.e. a mutable value cannot **escape** its ST

runST :: (**forall s.** ST s a) -> a s=any thread

e.g. factorial with mutable variable

```
import Data.STRef
factorial :: Int -> Int
factorial n = runST $ do
  x <- newSTRef 1
  -- x = mutable variable
  let f m
        | m <= 0 = pure ()
        | otherwise = modifySTRef x (* m) >> f (m -1)
  -- modify x
   in f n
  readSTRef x
```

```
```

==========
GHC.IORef

Library: https://hackage.haskell.org/package/base/docs/GHC-IORef.html

```
data IORef a

newIORef :: a -> IO (IORef a)
readIORef :: IORef a -> IO a
writeIORef :: IORef a -> a -> IO ()
modifyIORef :: IORef a -> (a -> a) -> IO ()
modifyIORef' :: IORef a -> (a -> a) -> IO ()
atomicModifyIORef :: IORef a -> (a -> (a,b)) -> IO b
makeWeakIORef :: IORef a -> IO () -> IO (Weak (IORef a))
```

IORef
A mutable value in IO
e.g.

```
test :: IO ()
test = do
  ref <- newIORef 1
  v <- readIORef ref             -- v=1
  writeIORef ref 10
  v <- readIORef ref             -- v=10
  modifyIORef ref (*10)
  v <- readIORef ref             -- v=100
```

==========
GHC.TypeLits

GHC.TypeLits: https://hackage.haskell.org/package/base/docs/GHC-TypeLits.html

```
-- GHC.TypeLits
-- Nat
data Nat
class KnownNat (n :: Nat)
natVal :: forall n proxy. KnownNat n => proxy n ->
Integer
natVal' :: forall n. KnownNat n => Proxy# n -> Integer

data SomeNat = forall (n :: Nat). KnownNat n => SomeNat
(Proxy n)
someNatVal :: Integer -> Maybe SomeNat

type (<=) x y = (x <=? y) ~ True
```

```
type family (m :: Nat) <=? (n :: Nat) :: Bool
type family (m :: Nat) + (n :: Nat) :: Nat
type family (m :: Nat) - (n :: Nat) :: Nat
type family (m :: Nat) * (n :: Nat) :: Nat
type family (m :: Nat) ^ (n :: Nat) :: Nat
type family Div (m :: Nat) (n :: Nat) :: Nat
type family Mod (m :: Nat) (n :: Nat) :: Nat
type family Log2 (m :: Nat) (n :: Nat) :: Nat
type family CompNat (m :: Nat) (n :: Nat) :: Ordering


-- Symbol
data Symbol
class KnownSymbol (s :: Symbol)
symbolVal :: forall s proxy. KnownSymbol => proxy s ->
String
symbolVal' :: forall s. KnownSymbol => Proxy# s ->
String

data SomeSymbol = forall (s :: Symbol). KnownSymbol s =>
SomeSymbol (Proxy s)
someSymbolVal :: String -> SomeSymbol

type family AppendSymbol (m :: Symbol) (n :: Symbol) ::
Symbol
type family CmpSymbol (m :: Symbol) (n :: Symbol) ::
Ordering


data ErrorMessage =
 Text Symbol
 | forall t. ShowType t
 | ErrorMessage :<>: ErrorMessage
    -- concat
 | ErrorMessage :$$: ErrorMessage
    -- newline

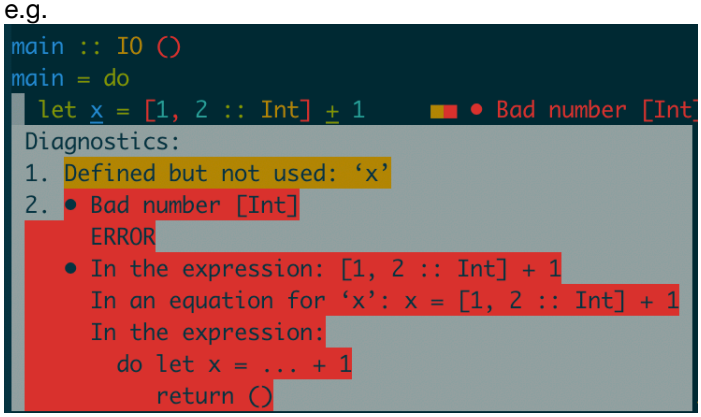
type family TypeError (a :: ErrorMessage) where
```

Usage: TypeError to provide error message of typical mistakes

e.g. common beginner mistake is adding a value to an array with +
 1 + [2, 3]

```
instance
  ( TypeError
      ( 'Text "Bad number [" :<>: ShowType a :<>: 'Text
"]"
          :$$: 'Text "ERROR" )
  ) =>
  Num [a]
```

e.g.



Usage: TypeError to forbid instances

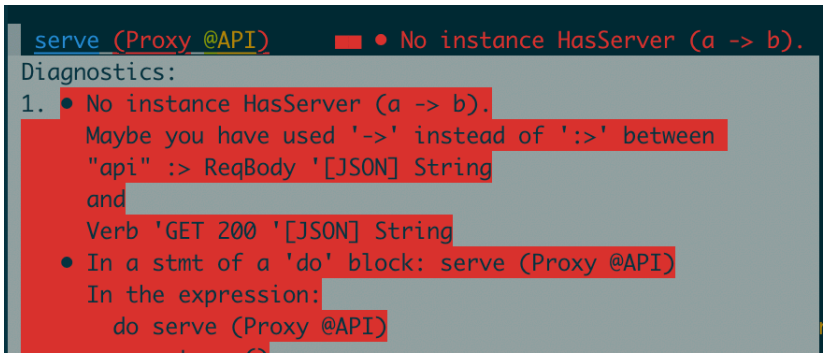
e.g. Servant prevents using -> instead of :> by mistake
https://hackage.haskell.org/package/servant-server/docs/Servant-Server.html

```
instance TypeError (HasServerArrowTypeError a b) =>
HasServer (a -> b) context
  where
    type ServerT (a -> b) m = TypeError
(HasServerArrowTypeError a b)
    route _ _ _ = error "servant-server panic:
impossible happened in HasServer (a -> b)"
    hoistServerWithContext _ _ _ = id

type HasServerArrowTypeError a b =
    'Text "No instance HasServer (a -> b)."
    ':$$: 'Text "Maybe you have used '->' instead of
':>' between "
    ':$$: 'ShowType a
    ':$$: 'Text "and"
    ':$$: 'ShowType b
```

Error

```
type API = "api" :> ReqBody '[JSON] String -> Get
'[JSON] String
```


serve (Proxy @API)
• No instance HasServer (a -> b).
Diagnostics:
1. • No instance HasServer (a -> b).
Maybe you have used '->' instead of ':>' between
"api" :> ReqBody '[JSON] String
and
Verb 'GET 200 '[JSON] String
• In a stmt of a 'do' block: serve (Proxy @API)
In the expression:
do serve (Proxy @API)